Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase II · Aprendizaje automático y redes neuronales

Módulo 9

Redes neuronales

Del perceptrón al modelo de lenguaje neuronal

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 9: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
$x \in \mathbb{R}^{1\times n}$	vector de entrada, escrito como fila
$W \in \mathbb{R}^{n\times m}$, $b \in \mathbb{R}^{1\times m}$	matriz de pesos y vector de sesgos de una capa $n \to m$
$z = xW + b$, $h = \phi(z)$	preactivación y activación de una capa
ϕ, ϕ'	función de activación, aplicada componente a componente, y su derivada
σ, Φ	sigmoide logística y función de distribución de la normal estándar
$\odot$	producto componente a componente (de Hadamard)
$p = \mathrm{softmax}(z)$, e_y	distribución de salida y vector *one-hot* de la clase y
$L = -\ln p_y$	pérdida de entropía cruzada de un ejemplo
$\nabla_W L$, δ	gradiente respecto a W (misma forma que W) y gradiente respecto a una preactivación
n_{in}, n_{out}	número de entradas y de salidas de una capa
$\mathcal{V}$, $K = \lvert\mathcal{V}\rvert$, $C \in \mathbb{R}^{K\times d}$	vocabulario, su tamaño y matriz de embeddings

A diferencia de los módulos 2 y 3, en este módulo los vectores se escriben como *filas* y las capas actúan por la derecha, $x \mapsto xW + b$. Es la convención de NumPy y PyTorch, la de la práctica del módulo y la de los artículos de Shazeer [18] y Touvron *et al.* [21]; con un lote de B ejemplos, x pasa a ser una matriz $B \times n$ y las fórmulas no cambian. Las clases y los tokens se numeran desde 0.

Índice

Sobre este resumen **ii**

1 Del perceptrón al perceptrón multicapa **1**
1.1 La unidad y el perceptrón . . . 1
1.2 La regla de aprendizaje de Rosenblatt . . . 1
1.3 El problema XOR . . . 2
1.4 Una capa oculta resuelve XOR . . . 2
1.5 El perceptrón multicapa . . . 2

2 Funciones de activación **4**
2.1 Sigmoide y tangente hiperbólica . . . 4
2.2 La unidad lineal rectificada . . . 4
2.3 Activaciones suaves: GELU y SiLU . . . 5
2.4 Unidades con puerta: GLU y SwiGLU . . . 5

3 Aproximación universal y profundidad **7**
3.1 Redes ReLU y funciones lineales a trozos . . . 7
3.2 El teorema en una dimensión . . . 7
3.3 El teorema general . . . 7
3.4 Qué no dice el teorema: el papel de la profundidad . . . 8

4 Inicialización y propagación de la varianza **9**
4.1 La ruptura de la simetría . . . 9
4.2 La varianza de una preactivación . . . 9
4.3 Activaciones lineales o tanh: LeCun y Xavier . . . 10
4.4 Activaciones ReLU: la inicialización de He . . . 10
4.5 Un experimento . . . 10

5 Retropropagación en un perceptrón multicapa **12**
5.1 Grafos computacionales . . . 12
5.2 El gradiente de la softmax con entropía cruzada . . . 12
5.3 Capas lineales y de activación . . . 13
5.4 Las ecuaciones de la retropropagación . . . 13
5.5 Un ejemplo completo . . . 13
5.6 Comprobación numérica del gradiente . . . 13

6 El modelo de lenguaje neuronal **15**

6.1 La maldición de la dimensionalidad 15

6.2 Embeddings como capa lineal 15

6.3 El modelo de Bengio *et al.* 16

6.4 Entrenamiento y regularización 16

6.5 Por qué generaliza 16

Bibliografía **18**

Tema 1

Del perceptrón al perceptrón multicapa

Este tema presenta la unidad básica de una red neuronal, el perceptrón, su regla de aprendizaje y el teorema que garantiza su convergencia, y muestra por qué una sola unidad no basta: la función XOR no es linealmente separable. La solución, añadir una capa oculta, conduce al perceptrón multicapa. La exposición sigue la §6.1 de [6] y las §6.1–6.3 de [10]; la demostración del teorema de convergencia sigue la lección 3 de [23], basada en Novikoff [13], y el recuento de parámetros, las notas «Neural Networks Part 1» de [27].

1.1 La unidad y el perceptrón

Definición 1.1 (Unidad). Una *unidad* (o *neurona artificial*) con n entradas está determinada por un vector de *pesos* $w \in \mathbb{R}^n$, un *sesgo* $b \in \mathbb{R}$ y una función $\phi\colon \mathbb{R} \to \mathbb{R}$, llamada *función de activación*. Para una entrada $x \in \mathbb{R}^n$ calcula la *preactivación* $z = w \cdot x + b$ y la *activación* $a = \phi(z)$.

Definición 1.2 (Perceptrón). El *perceptrón* de Rosenblatt [16] es la unidad con activación escalón:

$$\hat{y} = \mathbb{1}[\, w \cdot x + b > 0 \,] \in \{0, 1\}.$$

Su *frontera de decisión* es el hiperplano $\{x \in \mathbb{R}^n : w \cdot x + b = 0\}$.

El perceptrón es, por tanto, un clasificador lineal: asigna la clase 1 al semiespacio abierto $\{w \cdot x + b > 0\}$ y la clase 0 a su complementario.

Definición 1.4 (Separabilidad lineal). Un conjunto finito de pares (x_i, y_i) con $x_i \in \mathbb{R}^n$ e $y_i \in \{0, 1\}$ es *linealmente separable* si existen $w \in \mathbb{R}^n$ y $b \in \mathbb{R}$ tales que $w \cdot x_i + b > 0$ cuando $y_i = 1$ y $w \cdot x_i + b < 0$ cuando $y_i = 0$.

1.2 La regla de aprendizaje de Rosenblatt

Definición 1.5 (Regla del perceptrón). Sea $\eta > 0$ una *tasa de aprendizaje*. Se parte de $w = 0$, $b = 0$ y se recorren los ejemplos de forma cíclica. Para cada ejemplo (x_i, y_i) se calcula $\hat{y}_i$ y se actualiza

$$w \leftarrow w + \eta\,(y_i - \hat{y}_i)\,x_i, \qquad b \leftarrow b + \eta\,(y_i - \hat{y}_i).$$

El algoritmo termina cuando una pasada completa no produce ningún error.

Como $y_i - \hat{y}_i \in \{-1, 0, 1\}$, los pesos solo cambian cuando el ejemplo está mal clasificado, y el cambio desplaza la preactivación en la dirección correcta: si $y_i = 1$ y $\hat{y}_i = 0$, la nueva preactivación de x_i aumenta en $\eta(\|x_i\|^2 + 1) > 0$.

Para enunciar el teorema de convergencia conviene absorber el sesgo y usar etiquetas simétricas. Sea $\tilde{x}_i = (x_i, 1) \in \mathbb{R}^{n+1}$, $\theta = (w, b)$ y $s_i = 2y_i - 1 \in \{-1, +1\}$. El ejemplo i está bien clasificado si $s_i\,\theta \cdot \tilde{x}_i > 0$, y la regla de la definición 1.5 se escribe $\theta \leftarrow \theta + \eta\, s_i\, \tilde{x}_i$ cada vez que se comete un error. En particular, toda actualización se produce en un ejemplo con $s_i\,\theta \cdot \tilde{x}_i \le 0$.

Teorema 1.7 (Convergencia del perceptrón; Novikoff). *Supóngase que existen $u \in \mathbb{R}^{n+1}$ con $\|u\| = 1$ y $\gamma > 0$ tales que $s_i\, u \cdot \tilde{x}_i \ge \gamma$ para todo i, y sea $R = \max_i \|\tilde{x}_i\|$. Entonces, cualquiera que sea el orden en que se presentan los ejemplos, el número total M de actualizaciones de la regla del perceptrón cumple*

$$M \le \frac{R^2}{\gamma^2}.$$

Observación 1.8. La cota no depende de η ni del número de ejemplos, sino solo de la geometría de los datos: el cociente R/γ mide lo «estrecho» que es el pasillo que separa las dos clases. Si los datos no son linealmente separables, no existe tal u y la regla no termina nunca.

1.3 El problema XOR

La función XOR vale 1 en $(0,1)$ y $(1,0)$ y vale 0 en $(0,0)$ y $(1,1)$.

Proposición 1.9. *El conjunto $\{((0,0),0),((0,1),1),((1,0),1),((1,1),0)\}$ no es linealmente separable. En consecuencia, ningún perceptrón calcula XOR.*

Minsky y Papert [12] analizaron en 1969 esta y otras limitaciones de los perceptrones de una capa, y su libro contribuyó a frenar la investigación en redes neuronales durante más de una década [6, cap. 1].

1.4 Una capa oculta resuelve XOR

La salida consiste en componer dos capas. Sea $\mathrm{ReLU}(t) = \max(0, t)$ y considérese la red de la figura 1.1, tomada de [6, §6.1]:

$$h_1 = \mathrm{ReLU}(x_1 + x_2), \qquad h_2 = \mathrm{ReLU}(x_1 + x_2 - 1), \qquad \hat{y} = h_1 - 2h_2. \tag{1.1}$$

En el espacio de la capa oculta, las cuatro entradas se transforman en $(0,0)$, $(1,0)$, $(1,0)$ y $(2,1)$: las dos entradas con salida 1 caen en el mismo punto, y la función lineal $h_1 - 2h_2$ separa las clases. La capa oculta no clasifica: cambia la *representación* para que una función lineal pueda hacerlo. Aprender la representación a la vez que el clasificador es la idea que distingue el aprendizaje profundo de los modelos lineales del módulo 8.

1.5 El perceptrón multicapa

Definición 1.10 (Perceptrón multicapa). Sean $n_0, n_1, \dots, n_L$ enteros positivos. Un *perceptrón multicapa* (MLP) de L capas con anchuras $n_0 \to n_1 \to \cdots \to n_L$ está formado por matrices $W_\ell \in \mathbb{R}^{n_{\ell-1} \times n_\ell}$, sesgos $b_\ell \in \mathbb{R}^{1 \times n_\ell}$ y activaciones ϕ_ℓ, $\ell = 1, \dots, L$. Para $x \in \mathbb{R}^{1 \times n_0}$ se define $h^{(0)} = x$ y

$$z^{(\ell)} = h^{(\ell-1)} W_\ell + b_\ell, \qquad h^{(\ell)} = \phi_\ell\big(z^{(\ell)}\big), \qquad \ell = 1, \dots, L,$$

con ϕ_ℓ aplicada componente a componente. Las capas $1, \dots, L-1$ se llaman *ocultas*. En un clasificador de K clases, $n_L = K$, ϕ_L es la identidad y la salida $z^{(L)}$ es el vector de logits, que se convierte en $p = \mathrm{softmax}(z^{(L)})$.

Con un lote de B ejemplos apilados como filas de una matriz $X \in \mathbb{R}^{B \times n_0}$, la misma fórmula $XW_\ell + b_\ell$ (con b_ℓ sumado a cada fila) calcula todas las preactivaciones de la capa con un único producto de matrices. Esa es la razón práctica de la convención de vectores fila.

Proposición 1.11 (Número de parámetros). *Un MLP de anchuras $n_0 \to n_1 \to \cdots \to n_L$ tiene $\sum_{\ell=1}^{L} (n_{\ell-1} + 1)\, n_\ell$ parámetros.*

Proposición 1.13 (Colapso de las capas lineales). *Si todas las activaciones de un MLP son la identidad, existen $W \in \mathbb{R}^{n_0 \times n_L}$ y $b \in \mathbb{R}^{1 \times n_L}$ tales que $z^{(L)} = xW + b$ para toda entrada x. Además, $\operatorname{rg} W \leq \min_\ell n_\ell$.*

La proposición 1.13 explica por qué la no linealidad es imprescindible: sin ella, añadir capas no aumenta lo que la red puede representar, y una capa intermedia estrecha incluso lo reduce.

Tema 2

Funciones de activación

La activación determina qué funciones puede representar una red y cómo fluye el gradiente a través de ella. Este tema estudia las activaciones clásicas (sigmoide y tangente hiperbólica), la ReLU, que se impuso en la década de 2010, y las activaciones suaves y con puerta que usan los modelos de lenguaje actuales. Se sigue la §6.3 de [6], la §5.1.2 de [28] y las notas «Neural Networks Part 1» de [27]; las definiciones de GELU y SwiGLU proceden de los artículos originales de Hendrycks y Gimpel [8] y de Shazeer [18].

2.1 Sigmoide y tangente hiperbólica

Definición 2.1. La *sigmoide logística* y la *tangente hiperbólica* son

$$\sigma(t) = \frac{1}{1+e^{-t}}, \qquad \tanh(t) = \frac{e^{t}-e^{-t}}{e^{t}+e^{-t}}.$$

Proposición 2.2. *Para todo $t \in \mathbb{R}$:*

(i) $\sigma'(t) = \sigma(t)\big(1-\sigma(t)\big)$ *y* $0 < \sigma'(t) \le 1/4$, *con igualdad solo en* $t = 0$;

(ii) $\tanh(t) = 2\sigma(2t) - 1$;

(iii) $\tanh'(t) = 1 - \tanh^2(t) \in (0, 1]$.

Por (ii), una red con activaciones tanh y otra con sigmoides representan la misma familia de funciones: el factor 2 y el desplazamiento -1 se absorben en los pesos y sesgos de las capas adyacentes (ejercicio 2.3). La diferencia práctica es que tanh está centrada en 0.

Observación 2.3 (Saturación y desvanecimiento del gradiente). Cuando $|t|$ es grande, $\sigma'(t)$ y $\tanh'(t)$ son casi nulas: la unidad está *saturada*. En la retropropagación (tema 5) el gradiente que atraviesa una capa se multiplica por $\phi'(z)$; con sigmoides, tras k capas el factor es a lo sumo 4^{-k}, que para $k = 10$ vale menos de 10^{-6}. Este *desvanecimiento del gradiente* dificultó durante años el entrenamiento de redes profundas [6, §6.3.2].

2.2 La unidad lineal rectificada

Definición 2.4 (ReLU). La *unidad lineal rectificada* es $\mathrm{ReLU}(t) = \max(0, t)$. Su derivada es $\mathrm{ReLU}'(t) = \mathbb{1}[t > 0]$ para $t \neq 0$; en $t = 0$ se toma por convenio el valor 0.

La ReLU tiene tres ventajas [6, §6.3.1]: su cálculo es una comparación; en la región activa su derivada vale exactamente 1, de modo que el gradiente no se encoge al atravesar muchas capas; y es *positivamente homogénea*, $\mathrm{ReLU}(ct) = c\,\mathrm{ReLU}(t)$ para $c \ge 0$, lo que simplifica el análisis de la varianza (tema 4). Su inconveniente es que una unidad cuya preactivación es negativa para todos los datos tiene gradiente nulo y deja de aprender: se dice que está *muerta* [27]. Las

variantes *leaky* ReLU, $\max(\alpha t, t)$ con $0 < \alpha < 1$ fijo, y PReLU, con α aprendido [7], evitan ese problema.

2.3 Activaciones suaves: GELU y SiLU

Definición 2.5 (GELU y SiLU). Sea Φ la función de distribución de la normal estándar. Se definen

$$\mathrm{GELU}(t) = t\,\Phi(t), \qquad \mathrm{SiLU}(t) = t\,\sigma(t).$$

Más en general, $\mathrm{Swish}_\beta(t) = t\,\sigma(\beta t)$ con $\beta > 0$, de modo que $\mathrm{SiLU} = \mathrm{Swish}_1$.

Hendrycks y Gimpel [8] motivan GELU como el valor esperado de una puerta aleatoria: si $m \sim \mathrm{Bernoulli}(\Phi(t))$, entonces $\mathbb{E}[m\,t] = t\,\Phi(t)$. La entrada se deja pasar con una probabilidad que crece con su valor, en lugar de cortarse según su signo como en la ReLU. En la práctica se usa la aproximación

$$\mathrm{GELU}(t) \approx \tfrac{1}{2}\,t\Big(1 + \tanh\Big[\sqrt{2/\pi}\,\big(t + 0{,}044715\,t^3\big)\Big]\Big),$$

cuyo error absoluto es menor que $5 \cdot 10^{-4}$ en todo $\mathbb{R}$. GELU es la activación de BERT y de GPT-2.

Cuadro 2.1: Valores de ReLU, GELU y SiLU.

t	-2	-1	$-0{,}5$	0	$0{,}5$	1	2
$\mathrm{ReLU}(t)$	0	0	0	0	0,5	1	2
$\mathrm{GELU}(t)$	$-0{,}0455$	$-0{,}1587$	$-0{,}1543$	0	0,3457	0,8413	1,9545
$\mathrm{SiLU}(t)$	$-0{,}2384$	$-0{,}2689$	$-0{,}1888$	0	0,3112	0,7311	1,7616

Proposición 2.6. *(i)* $\mathrm{SiLU}'(t) = \sigma(t)\big(1 + t\,(1 - \sigma(t))\big)$*; en particular* $\mathrm{SiLU}'(0) = 1/2$*.*

(ii) $\mathrm{GELU}'(t) = \Phi(t) + t\,\varphi(t)$*, donde* φ *es la densidad normal estándar; en particular* $\mathrm{GELU}'(0) = 1/2$*.*

(iii) Ni GELU ni SiLU son monótonas: ambas tienen un único mínimo global negativo, que vale aproximadamente $-0{,}170$ *en* $t \approx -0{,}752$ *para GELU y* $-0{,}278$ *en* $t \approx -1{,}278$ *para SiLU.*

Cuando $\beta \to \infty$, $\sigma(\beta t) \to \mathbb{1}[t > 0]$ para $t \neq 0$ y Swish_β tiende a la ReLU; el ejercicio 2.7 prueba que la convergencia es uniforme.

2.4 Unidades con puerta: GLU y SwiGLU

En una unidad con puerta, una proyección de la entrada decide, componente a componente, cuánto se deja pasar de otra proyección.

Definición 2.7 (Capas prealimentadas con puerta; Shazeer). Sean $d, H \geq 1$, $W_g, W_u \in \mathbb{R}^{d\times H}$ y $W_d \in \mathbb{R}^{H\times d}$. Para $x \in \mathbb{R}^{1\times d}$ se definen, sin sesgos,

$$\mathrm{FFN}_{\mathrm{GLU}}(x) = \big(\sigma(xW_g) \odot xW_u\big)W_d, \qquad \mathrm{FFN}_{\mathrm{SwiGLU}}(x) = \big(\mathrm{SiLU}(xW_g) \odot xW_u\big)W_d.$$

La rama xW_g es la *puerta*, la rama xW_u es el *contenido* y W_d proyecta de vuelta a dimensión d. El MLP clásico de un Transformer es $\mathrm{FFN}(x) = \phi(xW_1)W_2$, con $W_1 \in \mathbb{R}^{d\times 4d}$ y $W_2 \in \mathbb{R}^{4d\times d}$.

La unidad GLU fue propuesta por Dauphin *et al.* para modelos de lenguaje convolucionales; Shazeer [18, §2] estudió variantes que sustituyen la sigmoide por otras funciones y observó que SwiGLU y GEGLU mejoran la calidad del Transformer frente a ReLU y GELU con el mismo número de parámetros.

Proposición 2.9 (Anchura de SwiGLU con igual número de parámetros). *Una capa* $\mathrm{FFN}_{\mathrm{SwiGLU}}$ *de anchura* H *tiene* $3dH$ *pesos y un MLP clásico* $d \to 4d \to d$ *tiene* $8d^2$*. Ambos coinciden si y solo si* $H = \frac{2}{3} \cdot 4d = 8d/3$*.*

Shazeer reduce la anchura en un factor 2/3 por este motivo [18, §2], y LLaMA adopta la misma regla: usa SwiGLU con dimensión oculta $\frac{2}{3}\,4d$ en lugar de $4d$ [21, §2.2]. Para LLaMA-7B, con $d = 4096$, se obtiene $8 \cdot 4096/3 \approx 10\,923$; el código publicado redondea al múltiplo de 256 siguiente, $H = 11\,008$, con $3 \cdot 4096 \cdot 11\,008 = 135\,266\,304$ pesos por capa frente a $2 \cdot 4096 \cdot 16\,384 = 134\,217\,728$ del MLP clásico. Qué activación usan internamente Claude, Gemini o Grok no es información pública.

Tema 3

Aproximación universal y profundidad

¿Qué funciones puede representar un perceptrón multicapa? Este tema demuestra que una sola capa oculta basta, en principio, para aproximar cualquier función continua en un compacto, y explica por qué ese resultado no justifica por sí solo el uso de redes anchas y poco profundas. La demostración en una dimensión es constructiva y usa solo la continuidad uniforme; el caso general se enuncia según Cybenko [4], Hornik [9] y Leshno *et al.* [11]. La discusión sobre la profundidad sigue la §6.4.1 de [6] y el artículo de Telgarsky [20]; la §5.1.1.4 de [28] y el apartado «Representational power» de [27] ofrecen el mismo punto de vista.

3.1 Redes ReLU y funciones lineales a trozos

Definición 3.1 (Función lineal a trozos). Una función continua $f\colon \mathbb{R} \to \mathbb{R}$ es *lineal a trozos* con *codos* $t_1 < \cdots < t_m$ si es afín en cada uno de los intervalos $(-\infty, t_1]$, $[t_1, t_2]$, …, $[t_m, \infty)$. Cada uno de esos $m+1$ intervalos es una *pieza*.

Proposición 3.2. *Sea $f(t) = c + \sum_{j=1}^{H} a_j \,\mathrm{ReLU}(w_j t + b_j)$ una red de una capa oculta con H unidades ReLU, entrada y salida escalares. Entonces f es continua y lineal a trozos con a lo sumo H codos, es decir, con a lo sumo $H+1$ piezas.*

Lema 3.3 (Sombrero). *La función $\Lambda(t) = \mathrm{ReLU}(t) - 2\,\mathrm{ReLU}(t-1) + \mathrm{ReLU}(t-2)$ vale 0 para $t \le 0$ y para $t \ge 2$, vale t en $[0,1]$ y $2 - t$ en $[1,2]$.*

Sumando sombreros desplazados y escalados se obtiene cualquier poligonal con nodos equiespaciados; el teorema siguiente usa una construcción equivalente y más económica.

3.2 El teorema en una dimensión

Teorema 3.4 (Aproximación universal en $\mathbb{R}$ con ReLU). *Sea $f\colon [a,b] \to \mathbb{R}$ continua y sea $\varepsilon > 0$. Existe una red $g(t) = c + \sum_{j=1}^{H} a_j \,\mathrm{ReLU}(t - \tau_j)$ con una capa oculta tal que $|f(t) - g(t)| < \varepsilon$ para todo $t \in [a,b]$.*

3.3 El teorema general

Teorema 3.6 (Aproximación universal; Cybenko, Hornik, Leshno *et al.*). *Sea $\phi\colon \mathbb{R} \to \mathbb{R}$ continua. Las funciones de la forma*

$$g(x) = \sum_{j=1}^{H} a_j \,\phi\big(x \cdot w_j + b_j\big), \qquad H \in \mathbb{N},\ a_j, b_j \in \mathbb{R},\ w_j \in \mathbb{R}^n,$$

son densas en el espacio de funciones continuas sobre cualquier compacto $K \subset \mathbb{R}^n$ con la norma del supremo si y solo si ϕ no es un polinomio.

Cybenko [4] lo demostró en 1989 para activaciones sigmoidales usando el teorema de Hahn–Banach y el de representación de Riesz; Hornik [9] lo extendió a activaciones acotadas y no constantes, y Leshno, Lin, Pinkus y Schocken [11] obtuvieron la caracterización anterior. La demostración del caso general excede el nivel de este curso. La necesidad de la condición sí es elemental:

Proposición 3.7. *Si ϕ es un polinomio de grado k, toda red de la forma del teorema 3.6 es un polinomio de grado a lo sumo k, y las redes no son densas en $C([0,1])$.*

3.4 Qué no dice el teorema: el papel de la profundidad

El teorema 3.6 es un resultado de existencia. No acota el número H de unidades necesarias, que para algunas funciones crece exponencialmente con la dimensión n de la entrada [6, §6.4.1]; no garantiza que el descenso de gradiente encuentre los pesos, y no dice nada sobre la generalización a datos no vistos. La profundidad aporta una ventaja que el teorema no recoge: componer capas permite reutilizar piezas, y algunas funciones que una red profunda representa con pocas unidades requieren exponencialmente muchas en una red poco profunda. El ejemplo siguiente lo muestra de forma exacta.

Proposición 3.8 (Diente de sierra). *Sea $g(t) = 2\,\mathrm{ReLU}(t) - 4\,\mathrm{ReLU}(t - \frac{1}{2})$, que en $[0,1]$ vale $2t$ en $[0, \frac{1}{2}]$ y $2 - 2t$ en $[\frac{1}{2}, 1]$, y sea $g^{(k)} = g \circ \cdots \circ g$ (k veces). Entonces:*

(i) en $[0,1]$, $g^{(k)}$ es lineal a trozos con exactamente 2^k piezas de longitud 2^{-k}, y en cada una recorre todo $[0,1]$;

(ii) $g^{(k)}$ se calcula con una red de k capas ocultas de 2 unidades ReLU cada una;

(iii) toda red de una capa oculta que coincida con $g^{(k)}$ en $[0,1]$ necesita al menos $2^k - 1$ unidades.

Con $k = 20$, la red profunda usa 40 unidades y la poco profunda más de un millón. Telgarsky [20] demostró que la separación persiste aunque solo se pida *aproximar*: hay funciones calculables por redes de profundidad del orden de k^3 y anchura constante que una red de profundidad $O(k)$ solo aproxima con del orden de 2^k unidades. Los LLM son profundos por esta razón: LLaMA-7B apila 32 bloques [21, §2], cada uno con un MLP de anchura moderada.

Tema 4

Inicialización y propagación de la varianza

El descenso de gradiente necesita un punto de partida. Si los pesos iniciales son demasiado grandes, las activaciones crecen capa a capa o saturan; si son demasiado pequeños, la señal se apaga y los gradientes con ella. Este tema muestra primero por qué los pesos no pueden inicializarse todos iguales y deduce después, a partir de la varianza de una suma de variables independientes (módulo 5), las reglas de inicialización de LeCun, Xavier y He. Se sigue la §8.4 de [6], la §5.4 de [28] y el apartado «Weight Initialization» de las notas «Neural Networks Part 2» de [27]; los resultados proceden de Glorot y Bengio [5] y de He *et al.* [7].

4.1 La ruptura de la simetría

Proposición 4.1 (Simetría de las unidades ocultas). *Sea un MLP con una capa oculta, $z = xW_1 + b_1$, $h = \phi(z)$, $o = hW_2 + b_2$, y sea L cualquier pérdida que dependa de o. Si en la inicialización las columnas i y j de W_1 son iguales, $(b_1)_i = (b_1)_j$, y las filas i y j de W_2 son iguales, entonces tras cualquier número de pasos de descenso de gradiente (por lotes o estocástico) esas igualdades se mantienen.*

En particular, si todos los pesos se inicializan a 0 (o a una misma constante), todas las unidades ocultas calculan siempre lo mismo y la capa equivale a una sola unidad [27], [28, §5.4.1.3]. Por eso los pesos se inicializan con valores *aleatorios*. Los sesgos, en cambio, pueden empezar en 0. En un modelo lineal, como la regresión logística, el problema no se plantea (ejercicio 4.1).

4.2 La varianza de una preactivación

Lema 4.2. *Sean w y h variables aleatorias independientes con $\mathbb{E}[w] = 0$. Entonces $\mathbb{E}[wh] = 0$ y $\operatorname{Var}(wh) = \operatorname{Var}(w)\,\mathbb{E}[h^2]$.*

Proposición 4.3 (Propagación hacia delante). *Sea $z = \sum_{i=1}^{n_{\text{in}}} w_i h_i$ la preactivación de una unidad. Supóngase que los w_i son independientes entre sí y de los h_i, con media 0 y varianza s^2, y que los h_i tienen el mismo segundo momento $\mathbb{E}[h^2]$. Entonces $\mathbb{E}[z] = 0$ y*

$$\operatorname{Var}(z) = n_{\text{in}}\, s^2\, \mathbb{E}[h^2].$$

El objetivo es elegir s^2 para que $\operatorname{Var}(z)$ sea la misma en todas las capas. Lo que relaciona $\mathbb{E}[h^2]$ con la varianza de la capa anterior es la activación.

4.3 Activaciones lineales o tanh: LeCun y Xavier

Si la activación es la identidad, o es tanh y opera cerca de 0, donde $\tanh t \approx t$, entonces $\mathbb{E}[h^2] \approx \mathrm{Var}(z_{\text{anterior}})$ y la proposición 4.3 da $\mathrm{Var}(z) = n_{\text{in}} s^2 \,\mathrm{Var}(z_{\text{anterior}})$. Conservar la varianza exige

$$s^2 = \frac{1}{n_{\text{in}}},$$

que es la inicialización de *LeCun*, la misma calibración $1/\sqrt{n}$ que recomiendan las notas de [27].

Proposición 4.4 (Propagación hacia atrás). *En la retropropagación (tema 5), el gradiente respecto a la activación h_i de una capa es $g_i = \sum_{j=1}^{n_{\text{out}}} W_{ij}\delta_j$. Con las mismas hipótesis de independencia y media nula,* $\mathrm{Var}(g_i) = n_{\text{out}}\, s^2\, \mathbb{E}[\delta^2]$.

Para que la varianza de los gradientes tampoco cambie de capa en capa hace falta $n_{\text{out}} s^2 = 1$. Las dos condiciones solo son compatibles si $n_{\text{in}} = n_{\text{out}}$; Glorot y Bengio [5] proponen el compromiso de tomar $1/s^2$ igual a la media aritmética de n_{in} y n_{out}.

Definición 4.5 (Inicialización de Xavier o Glorot). Los pesos de una capa $n_{\text{in}} \to n_{\text{out}}$ se toman independientes con media 0 y varianza

$$s^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}},$$

bien con distribución normal $\mathcal{N}(0, s^2)$, bien uniforme en $[-a, a]$ con $a = \sqrt{6/(n_{\text{in}} + n_{\text{out}})}$.

Lema 4.6. *Si $w \sim U(-a, a)$, entonces* $\mathrm{Var}(w) = a^2/3$.

Con $a^2 = 6/(n_{\text{in}} + n_{\text{out}})$, el lema 4.6 da $a^2/3 = 2/(n_{\text{in}} + n_{\text{out}})$, de modo que las dos versiones de la definición 4.5 tienen la misma varianza.

4.4 Activaciones ReLU: la inicialización de He

Lema 4.8. *Si z es una variable aleatoria con densidad simétrica respecto a 0 y varianza σ^2, entonces* $\mathbb{E}[\mathrm{ReLU}(z)^2] = \sigma^2/2$.

Teorema 4.9 (Inicialización de He). *En un MLP con activaciones ReLU, supóngase que las preactivaciones de cada capa tienen densidad simétrica y que se cumplen las hipótesis de la proposición 4.3. Entonces* $\mathrm{Var}(z^{(\ell+1)}) = \frac{1}{2}\, n_{\text{in}} s^2\, \mathrm{Var}(z^{(\ell)})$, *y la varianza se conserva capa a capa si y solo si*

$$s^2 = \frac{2}{n_{\text{in}}}.$$

He *et al.* [7] obtienen esta regla y la generalizan a PReLU; las notas de [27] la presentan como la recomendación práctica para redes ReLU. El factor 2 compensa exactamente que la ReLU anula la mitad del segundo momento. Con la inicialización de LeCun o de Xavier ($s^2 = 1/n$ cuando $n_{\text{in}} = n_{\text{out}} = n$), en cambio, la varianza se divide por 2 en cada capa ReLU y la desviación típica por $\sqrt{2}$.

4.5 Un experimento

El cuadro 4.1 muestra la desviación típica de las activaciones de una red de diez capas de anchura 256, con entradas normales estándar, para cinco inicializaciones (media de 20 repeti-

ciones con 1000 entradas cada una).

Cuadro 4.1: Desviación típica de las activaciones en una red de diez capas de anchura 256.

Pesos	Activación	Capa 1	Capa 4	Capa 10	Diagnóstico
$\mathcal{N}(0,1)$	tanh	0,975	0,974	0,974	saturada: 86,5 % con $\lvert h\rvert > 0{,}99$
$\mathcal{N}(0;\,0{,}01^2)$	tanh	0,156	$6{,}4 \cdot 10^{-4}$	$1{,}1 \cdot 10^{-8}$	señal apagada
Xavier	tanh	0,628	0,357	0,228	decae despacio
Xavier	ReLU	0,584	0,205	0,0254	se divide por $\sqrt{2}$ en cada capa
He	ReLU	0,826	0,819	0,812	estable

Los números concuerdan con la teoría. Con He, la preactivación de la capa 1 es $\mathcal{N}(0,2)$ y la desviación típica de $\mathrm{ReLU}(z)$ es $\sqrt{1-1/\pi} = 0{,}826$. Con Xavier y ReLU, el factor $2^{-1/2}$ por capa predice $0{,}584 \cdot 2^{-9/2} = 0{,}0258$ en la capa 10. Con $\mathcal{N}(0,1)$ y tanh la señal no desaparece, pero las unidades están saturadas: la derivada media $1 - h^2$ vale 0,051, y los gradientes se multiplican por números pequeños en cada capa (observación 2.3).

Tema 5

Retropropagación en un perceptrón multicapa

Entrenar una red por descenso de gradiente (módulo 7) exige calcular el gradiente de la pérdida respecto a todos los parámetros. La *retropropagación* lo hace aplicando la regla de la cadena del módulo 4 desde la salida hacia la entrada y reutilizando cada derivada intermedia. Este tema la deduce para un MLP con salida softmax y entropía cruzada y la aplica a mano a una red pequeña. Se sigue la §6.5 de [6], en particular la §6.5.4, las §6.6.2–6.6.4 de [10], el capítulo 8 de [2], la §5.3 de [28], las notas «Backpropagation, Intuitions» de [27] y la clase 3 de CS224N [24]. El método se popularizó con el artículo de Rumelhart, Hinton y Williams [17]; su generalización a grafos arbitrarios, la diferenciación automática, es el objeto del módulo 10.

5.1 Grafos computacionales

Definición 5.1 (Grafo computacional). Un *grafo computacional* es un grafo dirigido acíclico cuyos nodos son variables (escalares, vectores o matrices) y en el que cada nodo no inicial se obtiene aplicando una operación conocida a sus predecesores. La *pasada hacia delante* evalúa los nodos en un orden topológico; la *pasada hacia atrás* recorre el grafo en sentido inverso.

Proposición 5.2 (Regla de la cadena en un grafo). *Sea L un nodo escalar final y v un nodo con sucesores $u_1, \dots, u_r$. Entonces*

$$\frac{\partial L}{\partial v} = \sum_{k=1}^{r} \frac{\partial L}{\partial u_k} \frac{\partial u_k}{\partial v}.$$

La consecuencia algorítmica es que, si se conocen $\partial L/\partial u_k$ para todos los sucesores de v, el gradiente respecto a v se obtiene con operaciones locales. Recorriendo el grafo en orden topológico inverso, cada derivada se calcula una sola vez [6, §6.5.3]. Cuando un nodo alimenta a varios sucesores, sus contribuciones se *suman*.

5.2 El gradiente de la softmax con entropía cruzada

Proposición 5.3. *Sea $z \in \mathbb{R}^{1\times K}$, $p = \operatorname{softmax}(z)$ e $y \in \{0, \dots, K-1\}$, y sea $L = -\ln p_y$. Entonces*

$$\frac{\partial L}{\partial z_k} = p_k - \mathbb{1}[k = y], \qquad \text{es decir,} \qquad \nabla_z L = p - e_y.$$

Corolario 5.4. *Las componentes de $\nabla_z L$ suman 0.*

El gradiente tiene una lectura directa: el descenso sube el logit de la clase correcta, que es la única componente negativa, y baja los demás en proporción a la probabilidad que el modelo

les asignaba por error. La misma expresión $p - e_y$ es la que se retropropaga en cada posición durante el preentrenamiento de un LLM.

5.3 Capas lineales y de activación

Proposición 5.5 (Gradientes de una capa afín). *Sea $z = xW + b$ con $x \in \mathbb{R}^{1\times n}$, $W \in \mathbb{R}^{n\times m}$ y $b \in \mathbb{R}^{1\times m}$, y sea $g = \nabla_z L \in \mathbb{R}^{1\times m}$. Entonces*

$$\nabla_W L = x^\top g \in \mathbb{R}^{n\times m}, \qquad \nabla_b L = g, \qquad \nabla_x L = g\, W^\top \in \mathbb{R}^{1\times n}.$$

Para un lote $X \in \mathbb{R}^{B\times n}$ con gradientes $G \in \mathbb{R}^{B\times m}$ por filas y pérdida igual a la suma de las pérdidas, $\nabla_W L = X^\top G$, $\nabla_b L = \mathbf{1}^\top G$ y $\nabla_X L = GW^\top$.

Observación 5.6 (Análisis dimensional). El gradiente respecto a una matriz tiene siempre la forma de la matriz. En la proposición 5.5, $x^\top$ es $n \times 1$ y g es $1 \times m$, de modo que $x^\top g$ es $n \times m$, como W. Comprobar las formas basta a menudo para recordar el orden y las trasposiciones: «entrada traspuesta por gradiente de la salida» [27]. Con vectores columna, como en las notas de [27], las mismas fórmulas aparecen traspuestas.

Proposición 5.7 (Gradiente a través de una activación). *Sea $h = \phi(z)$ aplicada componente a componente. Entonces $\nabla_z L = \nabla_h L \odot \phi'(z)$.*

5.4 Las ecuaciones de la retropropagación

Teorema 5.8 (Retropropagación en un MLP de dos capas). *Sea el MLP $z^{(1)} = xW_1 + b_1$, $h = \phi(z^{(1)})$, $z^{(2)} = hW_2 + b_2$, $p = \mathrm{softmax}(z^{(2)})$, $L = -\ln p_y$. Definiendo $\delta^{(2)} = \nabla_{z^{(2)}} L$ y $\delta^{(1)} = \nabla_{z^{(1)}} L$, se cumple*

$$\begin{aligned}
&\delta^{(2)} = p - e_y, \quad && \nabla_{W_2} L = h^\top \delta^{(2)}, && \nabla_{b_2} L = \delta^{(2)},\\
&\nabla_h L = \delta^{(2)} W_2^\top, && \delta^{(1)} = \nabla_h L \odot \phi'(z^{(1)}), &&\\
&\nabla_{W_1} L = x^\top \delta^{(1)}, && \nabla_{b_1} L = \delta^{(1)}, && \nabla_x L = \delta^{(1)} W_1^\top.
\end{aligned}$$

Observación 5.9 (Coste). La pasada hacia delante realiza dos productos de matrices; la hacia atrás, tres para los gradientes de los parámetros y $\nabla_h L$, más uno opcional para $\nabla_x L$, todos de las mismas dimensiones. El coste de calcular el gradiente completo es, por tanto, del mismo orden que el de evaluar la red: aproximadamente el doble. Calcular cada derivada parcial por separado, con diferencias finitas, costaría una evaluación por parámetro.

5.5 Un ejemplo completo

5.6 Comprobación numérica del gradiente

Un error en una sola ecuación de la retropropagación produce gradientes incorrectos sin ningún mensaje de error. La comprobación estándar compara con diferencias finitas [27].

Proposición 5.11 (Diferencias centradas). *Sea $f\colon \mathbb{R} \to \mathbb{R}$ de clase C^3 en un entorno de θ. Entonces*

$$\frac{f(\theta+\varepsilon) - f(\theta-\varepsilon)}{2\varepsilon} = f'(\theta) + O(\varepsilon^2).$$

Aplicada a cada entrada de W_1 en el ejemplo 5.10 con $\varepsilon = 10^{-6}$, la fórmula reproduce $\nabla_{W_1} L$ con un error menor que 10^{-9}. La función ReLU no es derivable en 0; la comprobación es fiable mientras ninguna preactivación esté a distancia menor que ε de ese punto.

Tema 6

El modelo de lenguaje neuronal

Este tema reúne los anteriores en un modelo de lenguaje. Bengio, Ducharme, Vincent y Jauvin [1] propusieron en 2003 sustituir la tabla de un modelo de n-gramas por una red: cada token del contexto se convierte en un vector, los vectores se concatenan y un MLP produce la distribución del token siguiente. La presentación sigue la §2 del artículo original y la §6.5 de [10]; la motivación por el tamaño de las tablas, la §9.3 de [28]. Los resultados numéricos son los de la práctica del módulo, con el corpus de nombres del módulo 1.

6.1 La maldición de la dimensionalidad

Un modelo de n-gramas sobre un vocabulario de K tokens tiene K^n entradas (módulo 1, tema 2). Con $K = 28$ y un contexto de tres caracteres ($n = 4$), la tabla tiene $28^4 = 614\,656$ casillas, frente a los 1885 ejemplos de entrenamiento del corpus de nombres: casi todos los contextos no se han visto nunca y su estimación depende por completo del suavizado. El problema de fondo es que la tabla trata cada contexto como un símbolo aislado. Los contextos «.an» y «.en» no comparten nada, aunque las vocales `a` y `e` aparezcan en entornos parecidos.

6.2 Embeddings como capa lineal

Definición 6.1 (Matriz de embeddings). Sea $\mathcal{V}$ un vocabulario de K tokens y $d \geq 1$. Una *matriz de embeddings* es $C \in \mathbb{R}^{K \times d}$; el *embedding* del token de índice i es su fila $C[i] = e_i^\top C \in \mathbb{R}^{1 \times d}$, donde e_i es el vector *one-hot* de i (módulo 2, tema 5).

Proposición 6.2 (Gradiente de los embeddings). *Sea un contexto de tokens $(x_1, \ldots, x_c)$ y sea $e = (C[x_1], \ldots, C[x_c]) \in \mathbb{R}^{1 \times cd}$ la concatenación de sus embeddings. Si $\nabla_e L = (g_1, \ldots, g_c)$ con $g_j \in \mathbb{R}^{1 \times d}$, entonces*

$$\nabla_{C[i]} L = \sum_{j \,:\, x_j = i} g_j \qquad \text{para cada token } i,$$

y las filas de los tokens que no aparecen en el contexto tienen gradiente nulo.

En la práctica, las bibliotecas no multiplican por vectores *one-hot*: leen la fila correspondiente y, en la pasada hacia atrás, acumulan los gradientes en las filas usadas (en NumPy, con `np.add.at`).

6.3 El modelo de Bengio *et al.*

Definición 6.3 (Modelo de lenguaje neuronal prealimentado). Sea $n \geq 2$ el orden del modelo y $c = n - 1$ la longitud del contexto. Dado el contexto $(x_{t-c}, \ldots, x_{t-1})$, se calcula

$$e = \big(C[x_{t-1}], \ldots, C[x_{t-c}]\big) \in \mathbb{R}^{1\times cd}, \qquad y = b + eW + \tanh(\tilde{b} + eH)\,U \in \mathbb{R}^{1\times K},$$

con $H \in \mathbb{R}^{cd\times h}$, $\tilde{b} \in \mathbb{R}^{1\times h}$, $U \in \mathbb{R}^{h\times K}$, $b \in \mathbb{R}^{1\times K}$ y una matriz opcional de *conexiones directas* $W \in \mathbb{R}^{cd\times K}$, y se toma $P(x_t = k \mid \text{contexto}) = \operatorname{softmax}(y)_k$ [1, §2].

Proposición 6.4 (Número de parámetros). *Con conexiones directas, el modelo de la definición 6.3 tiene*

$$K(1 + nd + h) + h\big(1 + (n-1)d\big)$$

parámetros; sin ellas, $K(1 + nd + h) + h(1 + (n-1)d) - K(n-1)d$.

6.4 Entrenamiento y regularización

Los parámetros θ se ajustan minimizando la entropía cruzada media sobre los N pares (contexto, token siguiente) de entrenamiento, con decaimiento de pesos (módulo 7):

$$J(\theta) = -\frac{1}{N}\sum_{r=1}^{N} \ln P_\theta\big(x_t^{(r)} \mid \text{contexto}^{(r)}\big) + \frac{\lambda}{2}\|\theta\|^2, \qquad \theta \leftarrow \theta - \eta\big(\nabla\mathcal{L}(\theta) + \lambda\theta\big),$$

donde $\mathcal{L}$ es el primer término de J. El gradiente de $\mathcal{L}$ se obtiene con el teorema 5.8, aplicado al lote completo, y con la proposición 6.2 para C. La perplejidad es $\mathrm{PPL} = e^{\mathcal{L}}$ (módulo 1, tema 5).

Cuadro 6.1: Perplejidades en el corpus de nombres del módulo 1 (1000 pasos de descenso por lotes completos, $\eta = 0{,}5$).

Modelo	Contexto	Parámetros	PPL	
			entren.	valid.
Uniforme	0	0	28	28
Bigramas, suavizado $\alpha = 0{,}2$	1	784	—	12,42
MLP, $d = 10$, $H = 64$, $\lambda = 0{,}005$	3	4084	6,62	10,45
MLP sin decaimiento ($\lambda = 0$)	3	4084	3,76	30,73
Tabla de 4-gramas	3	614 656	—	(casi vacía)

El MLP regularizado reduce la perplejidad de validación del modelo de bigramas en un 16 %, de 12,42 a 10,45, con menos de una centésima parte de los parámetros de una tabla de 4-gramas. Sin decaimiento de pesos, la red memoriza los nombres de entrenamiento: la perplejidad de entrenamiento baja a 3,76 y la de validación sube a 30,73, peor que el modelo uniforme. Es el sobreajuste del módulo 8.

6.5 Por qué generaliza

La red no ve tokens, sino vectores. Si dos tokens tienen embeddings próximos, los contextos que difieren solo en ellos producen distribuciones próximas. El resultado siguiente lo cuantifica. Se usa la norma espectral $\|A\|_2 = \max_{\|v\|=1}\|vA\|$ (módulo 3).

Proposición 6.6. *Sea* $f(e) = \mathrm{ReLU}(eW_1 + b_1)W_2 + b_2$ *la función que lleva la concatenación de embeddings a los logits, y sea* $p(e) = \mathrm{softmax}(f(e))$*. Para todo par* e, e' *y todo token* k*,*

$$\|f(e) - f(e')\| \le \|W_1\|_2 \|W_2\|_2 \, \|e - e'\|, \qquad |\ln p_k(e) - \ln p_k(e')| \le 2 \, \|W_1\|_2 \|W_2\|_2 \, \|e - e'\|.$$

Los embeddings se aprenden junto con el resto de la red, y tokens que aparecen en contextos parecidos reciben gradientes parecidos (proposición 6.2); en la práctica del módulo, las vocales acaban agrupadas sin que nadie haya indicado a la red qué es una vocal. Lo aprendido para un contexto se transfiere así, por la proposición 6.6, a todos los contextos cercanos.

Bibliografía

[1] Y. Bengio, R. Ducharme, P. Vincent y C. Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003. https://www.jmlr.org/papers/v3/bengio03a.html

[2] C. M. Bishop y H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. Cap. 6, «Deep Neural Networks»; cap. 7, «Gradient Descent»; cap. 8, «Backpropagation». https://www.bishopbook.com/

[3] B. Raj *et al.* 11-785: Introduction to Deep Learning. Carnegie Mellon University, otoño de 2025. https://deeplearning.cs.cmu.edu/

[4] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.

[5] X. Glorot e Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. En *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, PMLR 9:249–256, 2010. https://proceedings.mlr.press/v9/glorot10a.html

[6] I. Goodfellow, Y. Bengio y A. Courville. *Deep Learning*. MIT Press, 2016. Cap. 6, «Deep Feedforward Networks», y §8.4, «Parameter Initialization Strategies». https://www.deeplearningbook.org/

[7] K. He, X. Zhang, S. Ren y J. Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. En *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015. https://arxiv.org/abs/1502.01852

[8] D. Hendrycks y K. Gimpel. Gaussian error linear units (GELUs). Prepublicación arXiv:1606.08415, 2016. https://arxiv.org/abs/1606.08415

[9] K. Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.

[10] D. Jurafsky y J. H. Martin. *Speech and Language Processing*, 3.ª ed., borrador del 19 de agosto de 2026. Cap. 6, «Neural Networks». https://web.stanford.edu/~jurafsky/slp3/

[11] M. Leshno, V. Y. Lin, A. Pinkus y S. Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867, 1993.

[12] M. Minsky y S. Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969.

[13] A. B. J. Novikoff. On convergence proofs on perceptrons. En *Proceedings of the Symposium on the Mathematical Theory of Automata*, vol. 12, págs. 615–622. Polytechnic Institute of Brooklyn, 1962.

[14] A. Radford, K. Narasimhan, T. Salimans e I. Sutskever. Improving language understanding by generative pre-training. Informe técnico, OpenAI, 2018. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

[15] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei e I. Sutskever. Language models are unsupervised multitask learners. Informe técnico, OpenAI, 2019. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[16] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

[17] D. E. Rumelhart, G. E. Hinton y R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[18] N. Shazeer. GLU variants improve Transformer. Prepublicación arXiv:2002.05202, 2020. https://arxiv.org/abs/2002.05202

[19] G. Strang. 18.065: Matrix Methods in Data Analysis, Signal Processing, and Machine Learning. Massachusetts Institute of Technology, MIT OpenCourseWare, primavera de 2018. Lección 27, «Backpropagation»; problemas de la sección VII.2 del libro *Linear Algebra and Learning from Data*. https://ocw.mit.edu/courses/18-065-matrix-methods-in-data-analysis-signal-processing-and-machine-learning-spring-2018/

[20] M. Telgarsky. Benefits of depth in neural networks. En *Conference on Learning Theory (COLT)*, 2016. https://arxiv.org/abs/1602.04485

[21] H. Touvron, T. Lavril, G. Izacard, X. Martinet *et al.* LLaMA: Open and efficient foundation language models. Prepublicación arXiv:2302.13971, 2023. https://arxiv.org/abs/2302.13971

[22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin. Attention is all you need. En *Advances in Neural Information Processing Systems 30*, 2017. https://arxiv.org/abs/1706.03762

[23] K. Q. Weinberger. CS4780/5780: Machine Learning for Intelligent Systems. Cornell University, otoño de 2018. Lección 3, «The Perceptron». https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote03.html

[24] D. Yang e Y. Choi. CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2026. Clase 3, «Backpropagation and Neural Network Basics», y clase 4, «Language Models and RNNs». https://web.stanford.edu/class/cs224n/

[25] C. Manning y R. Socher. CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2018. Tarea 1. https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1184/assignment1/assignment1.pdf

[26] Z. Yang, Z. Dai, R. Salakhutdinov y W. W. Cohen. Breaking the softmax bottleneck: A high-rank RNN language model. En *International Conference on Learning Representations (ICLR)*, 2018. https://arxiv.org/abs/1711.03953

[27] F.-F. Li *et al.* CS231n: Deep Learning for Computer Vision. Notas del curso. Stanford University, 2026. «Backpropagation, Intuitions» y «Neural Networks», partes 1 y 2. https://cs231n.github.io/

[28] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. Cap. 5, «Multilayer Perceptrons», y §9.3, «Language Models». https://d2l.ai/